I made a number of changes in response to reviewer comments; see below.

I made numerous grammar/spelling fixes.

I removed some text, as suggested by the reviewers. The page count (in Overleaf) decreased from 44 to 39.

Reviewer A:

This is an interesting paper that attempts to bridge from more abstract or theoretical work on models of music structure and nuance to something more practical, illustrated by some substantial examples of expressive performance achieved by manual specification. The work is not "complete" in the sense of being easily usable by a non-programmer or using off-the-shelf tools like score editors and sequencers, but it does point in a promising direction, backed by some examples. Therefore, I think this work is of interest to CMJ readers, the work is sufficiently novel (taking some steps beyond previous work that dates back at least to the 80's), and the work might challenge others to think about better ways to make more expressive and controllable inputs to control music synthesis.

There are a few shortcomings both in the presentation and in the library as described. The authors may object to any redesign of the library, but at least the presentation should be refined before publication.

1. The computational model is not clearly explained. In particular, time has multiple types or meanings that need to be clarified. There is score time, which appears to be measured in whole notes (I think most computational music theory would measure in quarters, but using 1/4 to represent one quarter certainly has its own logic). Then there is "adjusted time" or "real time" measured in seconds. However, multiple transformations can be applied, and it is unclear whether transformations map to score time or adjusted time. If score time, then tempo changes would have to map from metrical beats to real numbers that might as well be seconds. Then, to apply another transformation, the transformation could not be specified in terms of the original score time, which would be very difficult. Plus, the text says some transformations commute, which suggests (I'm guessing) that the transformations are not actually mapping from one time to another, but somehow altering attributes on events that remain in a score-time framework until all transformations have been applied. A more formal statement (precise description, equations) should be added to specify how the system works unambiguously.

I revised the 'MNM model' and 'Tempo control' sections to clarify the NMN timing model. The model can be summarized as follows:

- *Notes have start times and durations in both 'score time' and 'adjusted time' units. The score times never change. Initially the adjusted times are score times.*
- *A tempo adjustment is described by a tempo function defined over score time (not adjusted time!). A sequence of timing adjustments can be applied to a Configuration. Each adjustment modifies the adjusted times and durations of some or all notes.*
- *In the final Configuration, after all timing adjustments have been applied, adjusted time is the real time (in seconds) of events such as note starts and ends.*

A tempo adjustment maps one adjusted time system to another. But its definition doesn't refer to either of these systems; it refers only to score time.

There were a few places where I referred to 'time' without saying I meant 'adjusted' or 'score'. I fixed these.

2. It is mentioned that time transformations apply the average value of the function between two time points. How is the value averaged, noting that averaging in score time is different

from averaging in real time. Also, why average rather than compute the integral? I suppose they might be equivalent, depending on some details, but since these time/tempo/duration relationships were spelled out as early as Rogers' 1980 ICMC paper, you should clearly state that either you are following that model, or explain how you differ from the "standard model" and why.

The average value over an interval is the integral over the interval divided by the length of the interval.

Roger's paper describes a simple model in which the real-time duration of an interval is its score-time duration times the average of the 'slowness' (the reciprocal of tempo) over that interval. MNM generalizes this in several ways:

- *It allows pauses ('Dirac deltas') in the slowness function.*
- *It lets you specify rates of change as tempo, pseudo-tempo, or slowness.*
- *It defines the timing semantics when the tempo function is defined piecewise (with segment boundaries possibly between notes).*

3. Speaking of Rogers' paper, I think P. Batstone is 3rd author. Especially since this is a CMJ submission, another related work is Jaffe, D. 1985. "Ensemble Timing in Computer Music." Computer Music Journal 9(4):38-48. (Unfortunately, Jaffe apparently did not know of the earlier paper in ICMC.)

I meant to refer to Rogers and Rockstroh's earlier (1978) ICMC paper, of which Batstone is not an author; I fixed this.

I added a reference to Jaffe's paper, and discuss it in Related Work.

4. The model seems to arbitrarily treat virtual sustain pedals differently from standard pedals. This seems like a classic problem of mixing mechanism and policy. I would suggest that the theoretical model should offer unrestricted "mechanisms" with the same features in all pedals. This might even be simpler and makes aspects of the model more orthogonal and easier to explain. It may be that synthesizers or your implementation, as a matter of "policy" does not support certain pedal features, but it is often advantageous to separate these policy questions from the formal specification.

Both mechanisms use the word 'pedal' but they're different. Virtual sustain pedals let you delay the release of selected notes: for example, to sustain the notes in an arpeggio and have them released together. The synthesis engine is not involved.

On the other hand, the (non-virtual) sustain pedal tells the synthesis engine that the dampers are up, and that - for example - all strings resonate when a note is played.

5. The rule that "if a note at time T is shifted backward in time, pedals active at T are shifted backward by the same amount" suggests that there is something wrong with the formalization and some hack is necessary to make it work as desired. It seems that in most models, if time points are mapped to new times that all events will follow the mapping and no special rule should be necessary. I guess the situation here is that individual notes can be selected and moved in time, meaning that the resulting order of events is not necessarily maintained. This is both a powerful aspect of the model and something that should be clarified as perhaps an advantage over earlier models that all (I think) maintain event ordering. If I am not totally off-base in my understanding, it seems that you could have 2 notes at time T, with only one being shifted backward. Is there a problem that the pedal is following the timing of one note and not the other? Should pedals be attached to notes rather than times? If there is an inherent ambiguity or problem, you should at least mention it.

Yes, notes with the same score time can end up with different performance times: for

example, if Gaussian jitter is added to start times. And performance-time order can differ from score-time order. I added text to this effect.

The rule for pedal timing reflects musical pragmatics: if you pedal a chord at a particular time, you want it to 'catch' all the notes, including those that were moved back a millisecond by jitter. I don't think of this as a hack.

6. Defining dynamics in terms of MIDI velocity is expedient, but has real problems given the different interpretations of MIDI velocity across synthesizers. It would seem better to provide a more perception or physics-oriented semantics (e.g., see https://arxiv.org/pdf/2203.16294 or arXiv:2209.10674) and operations that have a more direct connection to the results than doing arithmetic on MIDI velocity.

MNM represents volumes as numbers in [0..1], not as MIDI velocities. They way that these numbers are mapped to synthesis parameters is in the realm of psychoacoustics, outside the scope of this paper.

Also it seems strange to represent volume in [0..1], but to implement VOL_SET with a parameter in [0..2].

Most volume adjustments are multiplicative, with factors typically in [0..2] (mapping the initial volume of .5 to [0..1]). So the toolkit defines constants ppp, mp, mf, ff etc. spanning the [0..2] range. For consistency, I wanted to use the same constants for adjustments using VOL_SET.

In conclusion, I reiterate that this work is of interest to CMJ readers, the work is sufficiently novel, and could hopefully inspire new work on interfaces for the specification of performance nuance. There are some shortcomings, particularly with a clear formalization of the model, and I think this should be improved before publication.

Reviewer D:

The authors describe a Python library for representing performance nuance. Although the main concepts described such as tempo transformations, mapping score time to actual time, and representing dynamics with piecewise functions are well known and established the work does unify them in an interesting way and having a scriptable, language based representation has advantages that alternatives do not have. I believe as a general topic this will be interesting to CMJ readers.

I have several suggestions/revisions that I believe should be addressed prior to publication:

1) The title is too general - I would suggest changing it to something more specific: A framework for modeling performance nuance in keyboard music

MNM isn't specific to keyboard music. It focuses on controlling note start/end times and note (onset) volumes. These parameters are relevant to a wide range of instruments.

I considered other titles:

- *'A framework for modeling performance nuance'. But 'framework' and 'model' sound a bit redundant.*
- *'A framework for expressing performance nuance'. But we discuss inferring nuance, as well as expressing it.*

Also, my intent is to focus on the model (layered adjustments, timing semantics, note selectors, PFTs) rather than the library. The library is one implementation; there could be others.

So I'd prefer to keep the title.

The current state of the library is very clearly focused to piano music

That's what I've used it for so far. But (except for pedal-related features) it can be used instruments other than piano, and for ensemble works. I added text clarifying this; it's also mentioned in the Future Work section.

2) Piecewise functions of time - there is a clear connection here with amplitude and control envelopes in Computer Music which are frequently specified this way. This should be acknowledged.

PFTs are more general than ADSR-type control envelopes:

- *They are used to describe lots of things: tempos, pauses, time shifts, pedal usages. Not just signal levels.*
- *They can have discontinuities and momentary values.*
- *They can be generated, manipulated and combined programmatically.*
- *The PFT model is an open framework; it allows arbitrary segment types to be added, as long as their values and derivatives can be computed.*

3) There are two areas of use suggested: nuance specification and nuance inference. For nuance specification the authors mention the target user is composers/musicians/musicologists. Currently there is no information about how the system would be received by other people than the authors. There is no information about how long it takes to create a nuance specification for a particular piece although there are hints that it can take significant effort and time.

For the nuance inference it is an interesting project and there is some existing work (see papers Rafael Ramirez, Esteban Maestre) but it is a challenging, open problem and the paper essentially speculates on how it could work without any empirical results. I suggest that the part of the paper discussing possible ways to do this without any grounding should be reduced to just mentioning that it is a possibility.

I did this, shrinking it to a paragraph and moving it to Future Work. I removed references to nuance inference in the Abstract and Introduction.

4) It is not clear to me how composability of transformations could work ? As the authors mention later in the paper some transformations need to be applied in a specific order and they provide some recommendations but I feel that the complexity of possible transformations could be very confusing to users of the system. I am also unclear if there is a fixed set of transformations that are applied or the sequence of transformation can be arbitrary. I think more concrete examples would help.

Arbitrarily many transformations can be applied; I added a sentence to this effect.

In some cases, transformations need to be done in a particular order. For example, the semantics of tempo adjustments require that adjusted times be an increasing function of score time. This means that if you want to change adjusted times (e.g. to create rolled chords, jitter, or agogic accents) this has to follow tempo adjustments. These rules are simple; I give them in the paper.

There are aspects of playing such as arpeggios, trills, vibrato that can be affected by some transformations but not others. The authors provide some limited references to vibrato at the later parts of the paper but this should be discussed in more detail earlier.

The paper doesn't deal with vibrato. The requirements of arpeggios etc. (at least those I

could think of) are met by MNM's features.

With virtual pedals you open the door for performances that are not realistic for actual piano performance. If the goal is to provide expressive control beyond what is possible by human players there are a lot of other ideas that have been explored in computer music that could be considered. This would increase the complexity of the system considerably.

The goal of the toolkit is to enable as wide a range of renditions as possible, including those that are physically impossible (or impossible for the particular performer).

It should be a straightforward extension to generalize the system to multiple pianos covering pieces for 4 hands or more.

Sure, or to handle multiple instruments, including non-keyboard instruments. I added text clarifying this.

The mapping of dynamics to MIDI velocity is simplistic. In practice different synthesizers handle this differently so allow for arbitrary mapping functions would be more general.

A mapping function could be inserted; that's outside of the scope. On the synthesizer I use (Pianoteq) this mapping is defined using a GUI.

How long does it take to annotate a piece ? What experience or background do the annotators need to have ? Many composers or performers are probably not comfortable with Python text programming. Even a pilot user study with some users beyond the authors would make the paper stronger. The authors mention a cycle of listening and annotating repeating thousands of times that seems very excessive.

A long and complex piece can have hundreds of nuance parameters, and each one may be adjusted several times. In the textual interface, this can take dozens of hours.

On one hand, this seems excessive. Hence I discuss more streamlined alternatives such as GUIs.

On the other hand, learning to play a piece on a physical instrument can take tens or hundreds of hours, and a large part of this time involves experimenting with nuance and figuring out how to physically realize it.

It would be good to provide some concrete examples in which the dynamic/scriptable nature of the representation compared to a static representation. There is some existing work on score representations that contrast dynamic score generation vs static.

Space limits don't allow showing a complete example. The final version will have links to examples, with both audio and source code, illustrating the value of scriptability.

There is a lot of future ideas for GUIs and applications that are not supported by any existing implementation. I would suggest reducing and condensing the amount of "future work" and focusing on the parts that are already implemented and used.

Actually I think the ideas about GUIs and applications are central to the paper.

As mentioned above, I condensed the 'Nuance Inference' section and moved it to 'Future Work'.

The authors should definitely mention RenCon (the performance rendering competition) that compares different computer systems for expressively rendering piano music based on transforming MIDI score representations as it is directly applicable to what is being proposed. It might be a good idea for the authors to check some of the participants and their

published work and explore how the proposed representation could be used to implement some of the systems described.

I hadn't heard of RenCon. I looked at the results of the most recent event (2025); there are 9 short papers: https://music-ir.org/mirex/wiki/2025:RenCon_Results

All of these involve algorithms (some AI-based, some not) for automatically adding nuance to a work described by a MusicXML file. I discuss such systems in Related Work. The RenCon projects have different goals than MNM, which is intended for humans to explicitly add nuance.